INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	*Computación gráfica, visión computacional y multimedia*				
TÍTULO DE LA PRÁCTICA:	*Transformaciones geométricas 3D*				
NÚMERO DE PRÁCTICA:	*03*	**AÑO LECTIVO:**	*2026-A*	**NRO. SEMESTRE:**	*VII*
FECHA DE PRESENTACIÓN	*03/05/2026*	**HORA DE PRESENTACIÓN**	*23:59 pm*		
INTEGRANTE (s): Perez Huamani Jeremy Joshua				**NOTA:**	
DOCENTE(s): Prof. René Nieto					

SOLUCIÓN Y RESULTADOS

I. EJERCICIOS PROPUESTOS

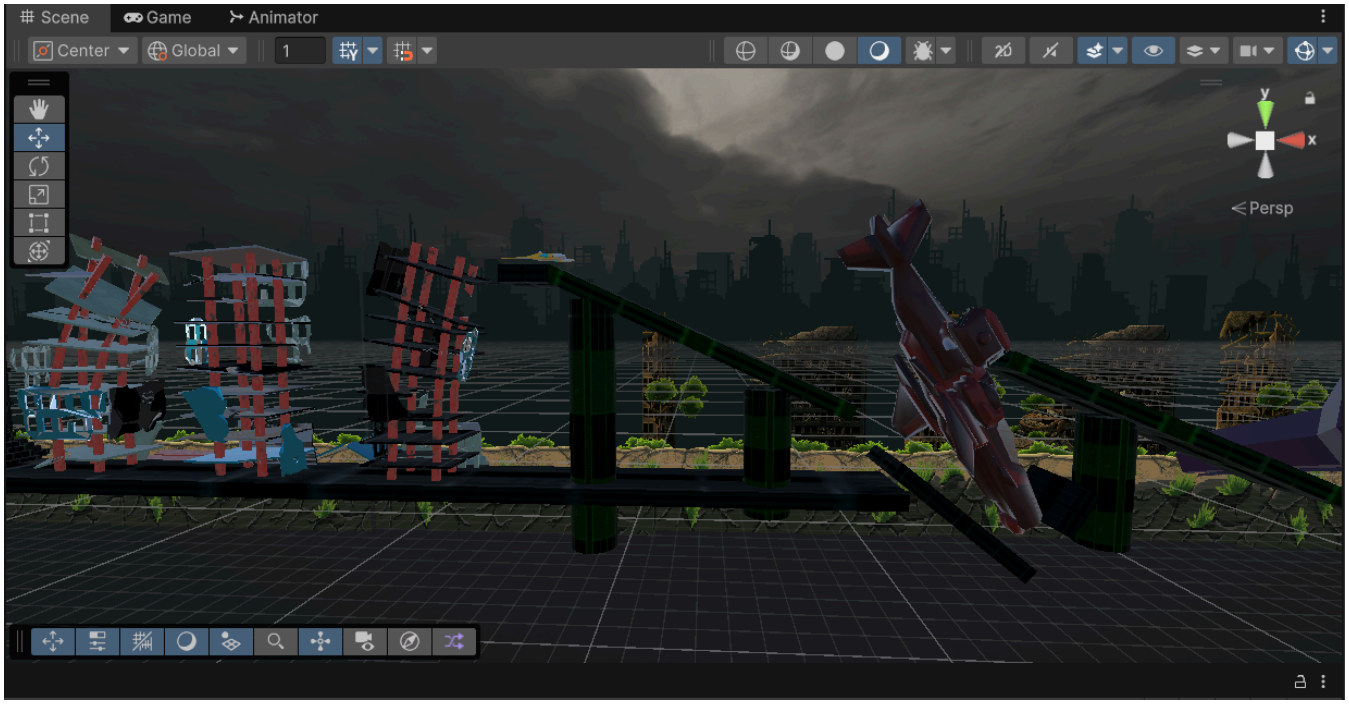
Carpeta con videos:

https://drive.google.com/drive/folders/145lXy7GcCjjkhQQFxo6mXWNVrOO4AO7F

La creación del nivel comenzó con la organización de la escena en Unity. Primero se definió el espacio principal del mapa, colocando una base o terreno sobre el cual el jugador pudiera desplazarse. Luego se añadieron diferentes objetos que representan la ciudad abandonada, tales como plataformas, edificios dañados, muros, estructuras metálicas, caminos y elementos decorativos.

El escenario fue construido de manera modular, es decir, utilizando varias piezas pequeñas que pueden repetirse o combinarse para formar un entorno más grande. Esta forma de trabajo permite modificar el nivel con mayor facilidad, ya que cada objeto puede moverse, duplicarse o ajustarse sin necesidad de rehacer toda la escena.

Además, se procuró que el diseño del nivel no sea únicamente visual, sino también jugable. Por eso, la distribución de los objetos se realizó considerando el movimiento del jugador, los espacios por donde puede avanzar, las zonas peligrosas y los puntos que marcan el progreso dentro del mapa.



Para facilitar el desarrollo del nivel, se utilizaron **prefabs**, que son objetos guardados en Unity como plantillas reutilizables. Estos permiten crear varias copias de un mismo elemento sin tener que configurarlo desde cero cada vez.

En el nivel de Ciudad Abandonada se utilizaron prefabs para elementos como:

- plataformas del escenario;
- estructuras urbanas;
- obstáculos;
- objetos decorativos;
- zonas peligrosas;
- puntos de finalización o transición del nivel;
- elementos relacionados con el jugador o la nave.

El uso de prefabs permitió mantener una mejor organización del proyecto, ya que cada objeto importante del nivel podía editarse desde su versión original. De esta manera, si se modificaba un material, una escala, una colisión o algún componente del prefab, los cambios podían aplicarse también a sus copias dentro de la escena.

Esto fue especialmente útil para construir una ciudad con varios elementos repetidos, como muros, suelos, estructuras rotas o partes del ambiente. Gracias a los prefabs, el nivel pudo desarrollarse de forma más rápida y ordenada.

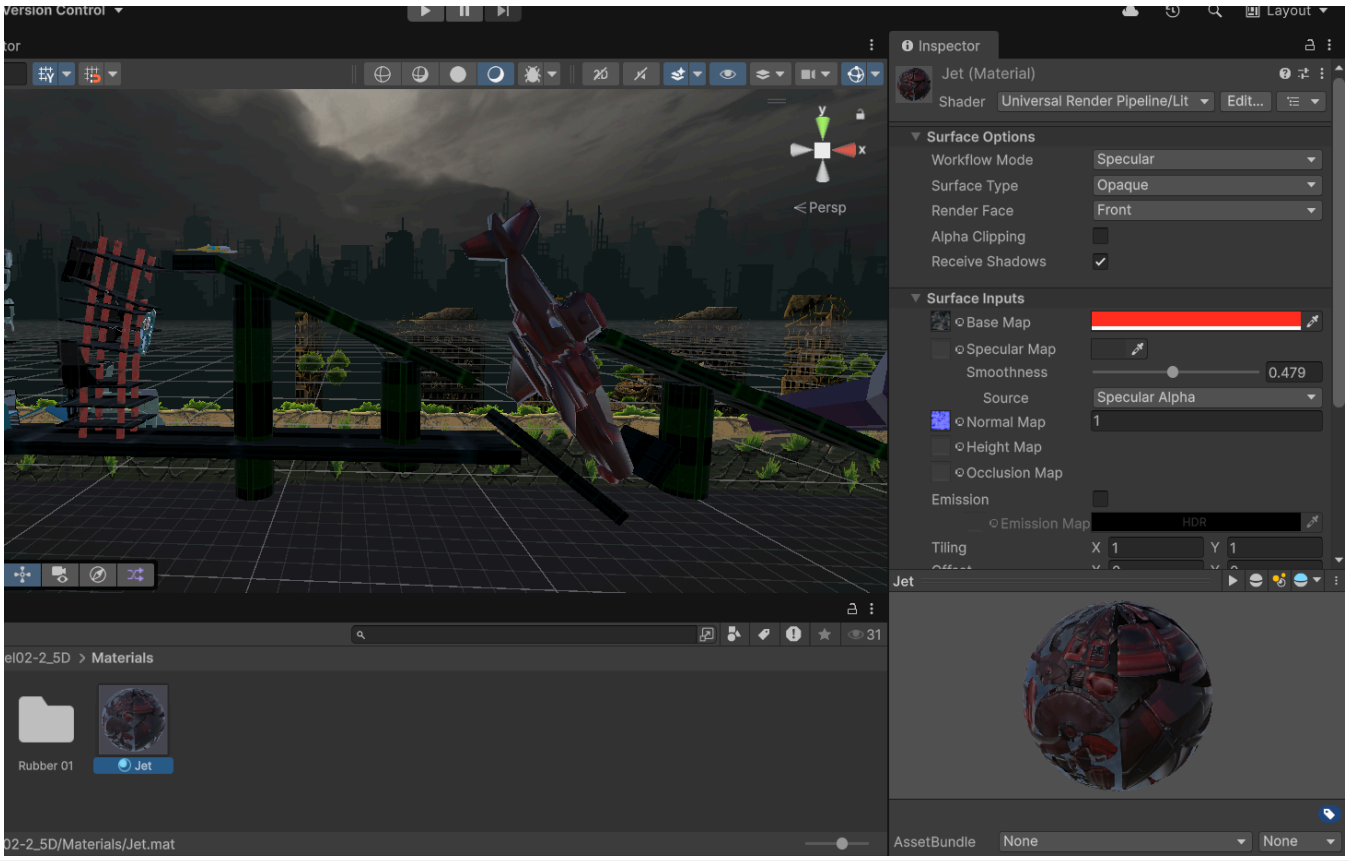
Los materiales fueron creados para darle identidad visual al nivel. En Unity, un material define cómo se ve la superficie de un objeto, incluyendo su color, textura, brillo, rugosidad y otros aspectos visuales.

Para el nivel de Ciudad Abandonada se crearon materiales relacionados con una estética deteriorada, utilizando colores oscuros, grises, marrones o metálicos, dependiendo del tipo de objeto. Por ejemplo, los edificios y muros utilizaron materiales con apariencia de concreto o superficie desgastada, mientras que otros objetos pudieron tener materiales metálicos o con tonos más sucios para reforzar la sensación de abandono.

El proceso general para crear los materiales fue el siguiente:

1. Se creó un nuevo material dentro de la carpeta del proyecto.
2. Se le asignó un nombre relacionado con el objeto o textura que representaba.
3. Se configuró el color base desde el apartado **Albedo** o **Base Map**.
4. En algunos casos, se añadieron texturas para mejorar el aspecto visual.
5. Se ajustaron propiedades como suavidad, brillo o apariencia metálica.
6. Finalmente, el material fue arrastrado hacia el objeto correspondiente dentro de la escena.

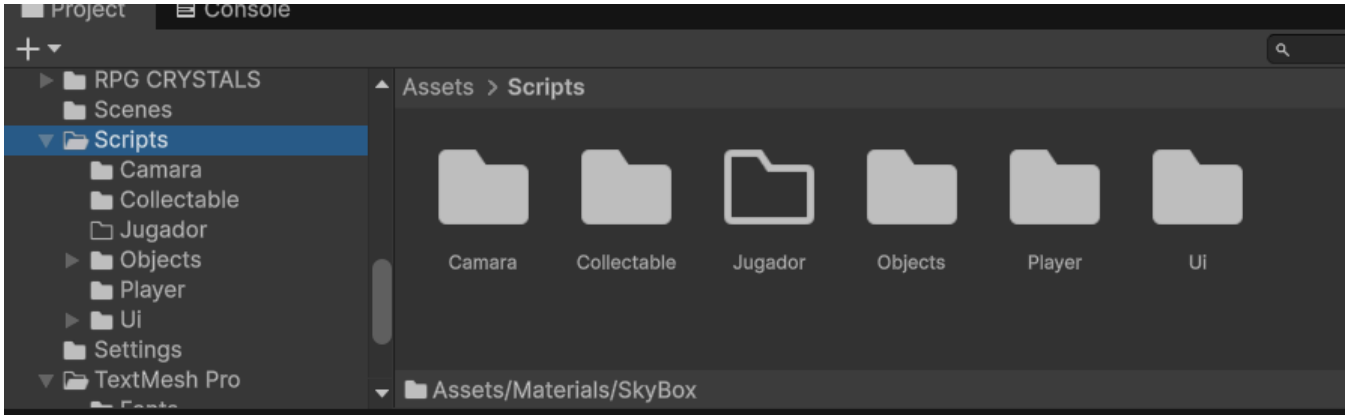
Este proceso permitió diferenciar visualmente cada parte del nivel. Por ejemplo, el suelo, los edificios, las plataformas y los objetos decorativos no se veían iguales, sino que tenían materiales propios, ayudando a que el escenario se percibiera más completo y coherente.



Para evitar desorden dentro del proyecto, los materiales se organizaron dentro de carpetas específicas, separándolos de los scripts, escenas, modelos y prefabs. Esto ayudó a trabajar de manera más clara, ya que cada recurso tenía una ubicación definida.

Además, al momento de aplicar materiales a los objetos, se revisó que cada modelo tuviera correctamente su componente **Mesh Renderer** o **Renderer**, ya que este componente permite visualizar el material asignado. En algunos casos, también fue necesario revisar que el objeto tuviera su malla correctamente configurada para que el material se mostrara de forma adecuada.

Esta parte fue importante porque, al importar modelos o prefabs externos, algunos objetos podían aparecer sin color o con materiales vacíos. Para solucionar esto, se asignaron manualmente los materiales correspondientes y se verificó que las texturas estuvieran bien conectadas.



El movimiento del jugador fue implementado mediante un script en C# dentro de Unity. Este script permite que el personaje o nave se desplace dentro del nivel según las teclas presionadas por el usuario.

El movimiento funciona leyendo las entradas del teclado en cada actualización del juego. Cuando el jugador presiona una tecla de dirección, el script interpreta esa entrada y aplica movimiento al objeto del jugador. Dependiendo de la configuración usada, este movimiento puede realizarse modificando directamente la posición del objeto mediante `transform`, o aplicando fuerza mediante un componente `Rigidbody`.

En este nivel, el movimiento está pensado para que el jugador pueda avanzar, retroceder y desplazarse por el escenario de forma controlada. Para ello, el objeto del jugador debe tener componentes importantes como:

- `Rigidbody`, para permitir comportamiento físico;
- `Collider`, para detectar choques con el escenario;
- script de movimiento, para procesar las entradas del teclado;
- cámara, si el seguimiento del jugador está configurado;
- audio o efectos, si se desea acompañar el movimiento con sonidos.

El uso del `Rigidbody` permite que el movimiento tenga relación con la física del juego, evitando que el jugador atraviese objetos sólidos y permitiendo que las colisiones se detecten correctamente. Además, se pueden

configurar restricciones para evitar rotaciones no deseadas, especialmente si el personaje o nave tiende a girar de forma extraña al chocar con otros objetos.

C/C++

```
using UnityEngine;
using UnityEngine.UI;

[RequireComponent(typeof(Rigidbody))]
public class SpaceshipController : MonoBehaviour
{
    [Header("Movement Settings")]
    [SerializeField] private float thrustForce = 10f;
    [SerializeField] private float rotationSpeed = 100f;
    [SerializeField] private float verticalThrustForce = 15f;
    [SerializeField] private float maxVelocity = 20f;

    [Header("Axis Settings")]
    [SerializeField] private Vector3 forwardAxis = Vector3.right; // Eje de movimiento
adelante/atrás
    [SerializeField] private Vector3 rotationAxis = Vector3.forward; // Eje de rotación
(Z para 2.5D)

    [Header("Fuel Settings")]
    [SerializeField] private float maxFuel = 100f;
    [SerializeField] private float fuelConsumptionRate = 5f;
    [SerializeField] private float verticalFuelConsumption = 10f;

    [Header("Health Settings")]
    [SerializeField] private float maxHealth = 100f;
    [SerializeField] private float damagePerCollision = 20f;

    [Header("UI References")]
    [SerializeField] private Slider fuelSlider;
    [SerializeField] private Slider healthSlider;

    [Header("Collision Tag")]
    [SerializeField] private string dangerousTag = "Dangerous";

    // Private variables
    private Rigidbody rb;
    private float currentFuel;
    private float currentHealth;
    private bool isAlive = true;

    void Start()
    {
        rb = GetComponent<Rigidbody>();

        // Initialize stats
```

```
        currentFuel = maxFuel;
        currentHealth = maxHealth;

        // Update UI
        UpdateUI();
    }

    void Update()
    {
        if (!isAlive) return;

        HandleRotation();
        HandleMovement();
        LimitVelocity();
        UpdateUI();

        // Check death condition
        if (currentHealth <= 0)
        {
            Die();
        }
    }

    void HandleRotation()
    {
        float rotation = 0f;

        // W: girar hacia arriba
        if (Input.GetKey(KeyCode.W))
        {
            rotation = rotationSpeed * Time.deltaTime;
        }

        // S: girar hacia abajo
        if (Input.GetKey(KeyCode.S))
        {
            rotation = -rotationSpeed * Time.deltaTime;
        }

        // Rotar en el eje Z para 2.5D
        transform.Rotate(rotationAxis, rotation);
    }

    void HandleMovement()
    {
        if (currentFuel <= 0)
        {
            // Sin combustible, no puede moverse
            return;
```

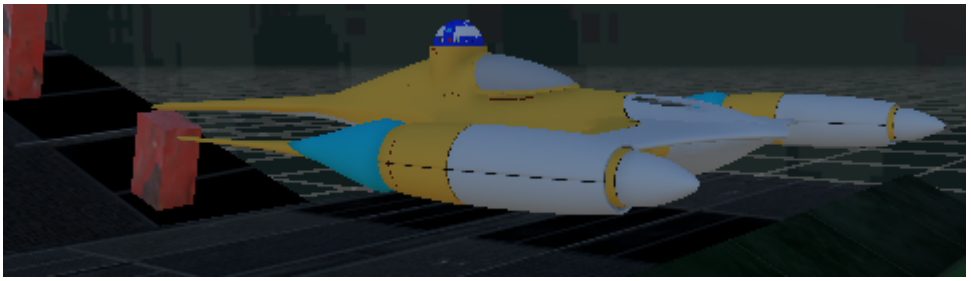
```
        }

        bool isMoving = false;

        // D: Mover hacia adelante (derecha en dirección local)
        if (Input.GetKey(KeyCode.D))
        {
            Vector3 force = transform.TransformDirection(forwardAxis) * thrustForce;
            rb.AddForce(force);
            isMoving = true;
        }

        // A: Mover hacia atrás (izquierda en dirección local)
        if (Input.GetKey(KeyCode.A))
        {
            Vector3 force = transform.TransformDirection(forwardAxis) * -thrustForce;
            rb.AddForce(force);
            isMoving = true;
        }

        // Space: Impulso vertical hacia arriba
        if (Input.GetKey(KeyCode.Space))
        {
            rb.AddForce(Vector3.up * verticalThrustForce);
            ConsumeFuel(verticalFuelConsumption * Time.deltaTime);
            isMoving = true;
        }

        // Consumir combustible si hay movimiento horizontal
        if (isMoving && !Input.GetKey(KeyCode.Space))
        {
            ConsumeFuel(fuelConsumptionRate * Time.deltaTime);
        }
    }

    void LimitVelocity()
    {
        if (rb.linearVelocity.magnitude > maxVelocity)
        {
            rb.linearVelocity = rb.linearVelocity.normalized * maxVelocity;
        }
    }

    void ConsumeFuel(float amount)
    {
        currentFuel = Mathf.Max(0, currentFuel - amount);
    }

    void TakeDamage(float damage)
```

```
{
    currentHealth = Mathf.Max(0, currentHealth - damage);
}

void UpdateUI()
{
    if (fuelSlider != null)
    {
        fuelSlider.value = currentFuel / maxFuel;
    }

    if (healthSlider != null)
    {
        healthSlider.value = currentHealth / maxHealth;
    }
}

void Die()
{
    isAlive = false;
    Debug.Log("Nave destruida!");
    // Aquí puedes agregar efectos de muerte, game over, etc.
    // Ejemplo: Destroy(gameObject);
    if (gameOverMenu != null)
    {
        gameOverMenu.TriggerGameOver();
    }
}

void OnCollisionEnter(Collision collision)
{
    if (collision.gameObject.CompareTag(dangerousTag))
    {
        TakeDamage(damagePerCollision);
        Debug.Log($"¡Colisión peligrosa! Vida restante: {currentHealth}");
    }
}

[Header("Box Limites")]
[SerializeField] private BoxCollider levelBounds;

private void LateUpdate()
{
    ClampToBounds();
}

private void ClampToBounds()
{
    if (levelBounds == null) return;
```

```
        Bounds b = levelBounds.bounds;
        Vector3 pos = transform.position;

        pos.x = Mathf.Clamp(pos.x, b.min.x, b.max.x);
        pos.y = Mathf.Clamp(pos.y, b.min.y, b.max.y);

        transform.position = pos;
    }

    // Método público para recargar combustible (útil para power-ups)
    public void RefillFuel(float amount)
    {
        currentFuel = Mathf.Min(maxFuel, currentFuel + amount);
    }

    // Método público para curar (útil para power-ups)
    public void Heal(float amount)
    {
        currentHealth = Mathf.Min(maxHealth, currentHealth + amount);
    }

    // Getters públicos
    public float GetCurrentFuel() => currentFuel;
    public float GetCurrentHealth() => currentHealth;
    public bool IsAlive() => isAlive;

    [SerializeField] private GameOverMenuController gameOverMenu;

}
```

Dentro del nivel también se configuraron colisiones para que el jugador no atraviese los objetos del escenario. Para ello, los elementos principales del mapa cuentan con componentes Collider, como Box Collider, Mesh Collider u otros, dependiendo de la forma del objeto.

Estas etiquetas permiten que los scripts reconozcan qué tipo de objeto se está tocando. Por ejemplo, si el jugador colisiona con una zona marcada como Peligroso, se puede reiniciar la escena. Si toca un objeto con la etiqueta Final, se puede cargar el siguiente nivel.



Con esto podemos ver la rotación y traslación de objetos 3D en unity.

II. SOLUCIÓN DEL CUESTIONARIO

No hay un cuestionario

III. CONCLUSIONES

- El uso de Parallax permite que un escenario 2D se vea más profundo y dinámico, ya que los fondos se mueven a distintas velocidades según su distancia aparente.
- Esta técnica ayuda a que el jugador sienta mejor el movimiento dentro del nivel, porque no solo se desplaza el personaje, sino también el entorno de una forma más natural.
- En Unity, el Parallax es útil porque mejora bastante la parte visual del juego sin necesidad de usar modelos 3D complejos.
- También permite separar el escenario en capas, como fondo lejano, montañas, árboles o nubes, haciendo que cada elemento tenga una función visual dentro de la escena.
- Una buena implementación de Parallax puede hacer que el nivel se sienta más vivo, aunque el juego siga siendo principalmente 2D.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[]https://www.youtube.com/watch?v=GbmRt0wydQU
[]https://didigameboy.itch.io/jambo-jungle-free-sprites-asset-pack/download/eyJleHBpcmVzIjoxNzc2ODI3NDAxLCJpZCI6MTUxNzE0fQ%3d%3d%2eZOk0YHJ7il1eY2vFT9pM9e1%2fh1M%3d